



ANNA UNIVERSITY, CHENNAI

KARPAGA VINAYAGA COLLEGE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, Affiliated to Anna University)

GST Road, Karpaga Vinayaga Nagar,
Palayanoor Post, Madhuranthagam Taluk,
Chengalpattu Dist-603308.

LABORATORY RECORD



FACULTY OF INFORMATION AND COMMUNICATION ENGINEERING

(MASTER OF COMPUTER APPLICATIONS)

M.C.A

MC25106 – ADVANCED JAVA PROGRAMMING LABORATORY

REG.NO :

NAME :

YEAR : I - MCA

SEMESTER : I

Karpaga Vinayaga College of Engineering and Technology

DEPARTMENT OF COMPUTER APPLICATIONS



Name :

Reg. No :

--	--	--	--	--	--	--	--	--	--	--	--

Subject Code : MC25106

Subject : ADVANCED JAVA PROGRAMMING LABORATORY

Course : MASTER OF COMPUTER APPLICATIONS

Certified that this is the bonafide record of practicals done as a part of 1st semester during the academic year 2025 - 2026

Staff In-charge

Head of the Department

Submitted for University Practical Examination held on _____

Internal Examiner

External Examiner

INDEX

Ex.No	Date	Name of the Experiment	Page No	Mark	Sign
1.		Write a Java program that demonstrates the use of classes, objects, constructors, method overloading, inheritance, super, and abstract classes.			
2.		Create a Java application that includes built-in and user-defined exception handling using try, catch, finally, throw, and throws.			
3.		Implement multiple threads in Java using the Thread class and Runnable interface. Demonstrate thread lifecycle, priorities, and synchronization.			
4.		Connect to a relational database (MySQL/PostgreSQL) and perform Create, Read, Update, and Delete operations using JDBC			
5.		Implement a simple servlet for uploading files to the server and downloading them using Multipart Config and File Output Stream.			
6.		Develop a servlet to handle HTTP GET and POST methods. Accept form data (e.g., user registration) and display it			
7.		Design a responsive UI form using Bootstrap and React to capture user input and display the result			
8.		Create a JSP application using scripting elements, EL for data display, and JSTL for conditional and loop-based rendering			
9.		Build a Java application using Hibernate and JPA annotations to perform CRUD operations on a Student entity.			
10.		Mini-project integrating Spring and Hibernate framework			

Ex.No:01

Write a Java program that demonstrates the use of classes, objects, constructors, method overloading, inheritance, super, and abstract classes.

Aim

To write a Java program that demonstrates the use of **classes, objects, constructors, method overloading, inheritance, super keyword, and abstract classes.**

Procedure

Step 1: Create an **abstract class** named Animal with:

- A constructor.
- An abstract method sound().
- A normal method eat().

Step 2: Create a **derived class** Dog that extends Animal.

Step 3: Use the **super** keyword to call the parent class constructor.

Step 4: Implement the abstract method sound().

Step 5: Demonstrate **method overloading** by creating two play() methods with different parameters.

Step 6: In the **main class**, create an **object** of Dog and call the methods.

Step 7: Observe how inheritance, method overloading, and abstract class concepts work together.

Program

```
// Abstract class abstract class Animal{
String name;
// Constructor Animal(String name){
this.name=name;
System.out.println("Animal constructor called for:"+ name);
}
// Abstract method abstract void sound();
//Normal method void eat() {
System.out.println(name+"is eating.");
}
}
// Subclass (Inheritance) class Dog extends Animal{
```

```

int age;

//Constructor using super keyword Dog(String name, int age) {
super(name);//calling parent constructor this.age = age;
System.out.println("Dog constructor called.Age:"+age);
}

//Implementing abstract method void sound() {
System.out.println(name+"says: Woof! Woof!");
}

//Method Overloading void play() {
System.out.println(name+"is playing.");
}

Void play(String toy){
System.out.println(name+"is playing with a"+ toy+".");
}
}

//Main class
Public class Main Class{
public static void main(String[] args){
//Creating object(Class &Object demo) Dog d1 = new Dog("Buddy", 3); d1.eat(); // Inherited method
d1.sound();           //Overridden abstract method d1.play();      // Overloaded method
d1.play("ball");//Overloaded method with argument
}
}

```

Output

Animal constructor called for: Buddy Dog
constructor called. Age: 3 Buddy is eating.
Buddy says: Woof! Woof! Buddy is playing.
Buddy is playing with a ball.

Result

Thus, a Java program was successfully written and executed to demonstrate the use of **classes, objects, constructors, method overloading, inheritance, the super keyword, and abstract classes**

Ex. No : 02

Date:

Create a Java application that includes built-in and user-defined exception handling using try, catch, finally, throw, and throws.

Aim

To Create a Java application that includes built-in and user-defined exception handling using try, catch, finally, throw, and throws

Algorithm

Step 1: Start the program

Step 2: Inside main(), try dividing a number by zero

Step 3: Catch the built-in ArithmeticException

Step 4: Execute the finally block

Step 5: Call checkAge() which uses throws

Step 6: In checkAge(), if age < 18 → use throw to create user-defined exception

Step 7: Catch the user-defined exception in main()

Step 8: Execute its finally block

Step 9: End program

Program

```
// SimpleExceptionApp.java
```

```
// User-defined exception
```

```
class InvalidAgeException extends Exception {
```

```
    public InvalidAgeException(String msg) {
```

```
        super(msg);
```

```
    }
```

```
}
```

```
public class SimpleExceptionApp {
```

```
    // Method with 'throws'
```

```
    static void checkAge(int age) throws InvalidAgeException {
```

```
        if (age < 18) {
```

```
            // Using 'throw'
```

```

        throw new InvalidAgeException("Age must be 18 or above!");
    }
    System.out.println("Age is valid.");
}
public static void main(String[] args) {
    // === Built-in Exception Example ===
    System.out.println("Built-in Exception Example:");
    try {
        int x = 10 / 0; // built-in exception
    } catch (ArithmeticException e) {
        System.out.println("Caught built-in exception: " + e.getMessage());
    } finally {
        System.out.println("Finally block for built-in exception.");
    }
    // === User-defined Exception Example ===
    System.out.println("\nUser-defined Exception Example:");
    try {
        checkAge(15); // this triggers user-defined exception
    } catch (InvalidAgeException e) {
        System.out.println("Caught user-defined exception: " + e.getMessage());
    } finally {
        System.out.println("Finally block for user-defined exception.");
    }

    System.out.println("\nProgram Ended.");
}
}

```


Output

Built-in Exception Example:

Caught built-in exception: / by zero

Finally block for built-in exception.

User-defined Exception Example:

Caught user-defined exception: Age must be 18 or above!

Finally block for user-defined exception.

Program Ended.

Result

Thus the above program includes built-in and user-defined exception handling using try, catch, finally, throw, and throws was executed successfully.

Ex. No : 03

Date:

Implement multiple threads in Java using the Thread class and Runnable interface. Demonstrate thread lifecycle, priorities, and synchronization.

Aim

To Implement multiple threads in Java using the Thread class and Runnable interface. Demonstrate thread lifecycle, priorities, and synchronization.

Algorithm

1. Start the program
2. Create a shared class `Counter` with a **synchronized** `increment()` method
3. Create a class `MyThread` extending **Thread class**
4. Create a class `MyRunnable` implementing **Runnable interface**
5. In both run methods:
 - Display thread running
 - Increment shared counter
 - Sleep to show thread lifecycle (Running → Timed Waiting → Running)
6. In `main()`
 - Create one `MyThread` object
 - Create one `Thread` object using `Runnable`
 - Set thread names and priorities
 - Start both threads
 - Wait for threads using `join()`
7. Display final counter value
8. End the program

Program

```
// MultiThreadProgram.java
// Shared synchronized counter
class Counter {
    private int value = 0;
    public synchronized void increment() {
        value++;
    }
    public int getValue() {
        return value;
    }
}
```

```

    }
}
// Thread class example
class MyThread extends Thread {
    Counter counter;
    MyThread(Counter c) {
        counter = c;
    }
    public void run() {
        System.out.println(getName() + " → RUNNING");
        for (int i = 0; i < 3; i++) {
            counter.increment();
            System.out.println(getName() + " → Counter = " + counter.getValue());
            try {
                Thread.sleep(500); // Timed Waiting
            } catch (InterruptedException e) {
            }
        }
        System.out.println(getName() + " → FINISHED");
    }
}
// Runnable interface example
class MyRunnable implements Runnable {
    Counter counter;
    MyRunnable(Counter c) {
        counter = c;
    }
    public void run() {
        String name = Thread.currentThread().getName();
        System.out.println(name + " → RUNNING");
        for (int i = 0; i < 3; i++) {
            counter.increment();
            System.out.println(name + " → Counter = " + counter.getValue());

```

```

        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
        }
    }
    System.out.println(name + " → FINISHED");
}
}

public class MultiThreadProgram {
    public static void main(String[] args) {
        Counter counter = new Counter();
        // Creating threads
        MyThread t1 = new MyThread(counter);
        Thread t2 = new Thread(new MyRunnable(counter));

        // Setting thread names
        t1.setName("Thread-A");
        t2.setName("Thread-B");
        // Setting priorities
        t1.setPriority(Thread.MAX_PRIORITY); // 10
        t2.setPriority(Thread.MIN_PRIORITY); // 1
        System.out.println("Starting Threads...\n");
        // Starting threads (New → Runnable → Running)
        t1.start();
        t2.start();
        try {
            t1.join();
            t2.join();
        } catch (Exception e) {}
        System.out.println("\nFINAL Counter Value = " + counter.getValue());
        System.out.println("Main Thread Finished.");
    }
}

```

```
} }
```

Output

Starting Threads...

Thread-A → RUNNING

Thread-A → Counter = 1

Thread-B → RUNNING

Thread-B → Counter = 2

Thread-A → Counter = 3

Thread-B → Counter = 4

Thread-A → Counter = 5

Thread-B → Counter = 6

Thread-A → FINISHED

Thread-B → FINISHED

FINAL Counter Value = 6

Main Thread Finished.

Result

Thus the above program multiple threads in Java using the Thread class and Runnable interface. Demonstrate thread lifecycle, priorities, and synchronization was executed successfully.

Ex. No : 04

Date:

Connect to a relational database (MySQL/PostgreSQL) and perform Create, Read, Update, and Delete operations using JDBC

Aim

To write a program to **Connect** to a relational database (MySQL/PostgreSQL) and perform Create, Read, Update, and Delete operations using JDBC

Algorithm

1. **Load JDBC Driver** using `Class.forName()`.
2. **Establish Connection** using `DriverManager.getConnection()`.
3. **Create a Statement / PreparedStatement** for executing SQL queries.
4. **CREATE Operation:** Insert a new record into the database.
5. **READ Operation:** Fetch and display all records.
6. **UPDATE Operation:** Modify a record based on ID.
7. **DELETE Operation:** Remove a record from the table.
8. **Close the Connection.**

Program

```
import java.sql.*;

public class JDBCCrudExample {

    public static void main(String[] args) {

        // Database URL, username, password
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "your_password";

        try {

            // 1. Load Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // 2. Establish Connection
            Connection con = DriverManager.getConnection(url, user, password);
```

```

System.out.println("Connected to Database!\n");
// ----- CREATE -----
String insertQuery = "INSERT INTO students VALUES (?, ?, ?)";
PreparedStatement psInsert = con.prepareStatement(insertQuery);
psInsert.setInt(1, 1);
psInsert.setString(2, "Alice");
psInsert.setInt(3, 20);
psInsert.executeUpdate();
System.out.println("Record Inserted!");
// ----- READ -----
System.out.println("\nReading Records:");
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
while (rs.next()) {
    System.out.println(rs.getInt("id") + " | " +
        rs.getString("name") + " | " +
        rs.getInt("age"));
}
// ----- UPDATE -----
String updateQuery = "UPDATE students SET age = ? WHERE id = ?";
PreparedStatement psUpdate = con.prepareStatement(updateQuery);
psUpdate.setInt(1, 22);
psUpdate.setInt(2, 1);
psUpdate.executeUpdate();
System.out.println("\nRecord Updated!");
// ----- DELETE -----
String deleteQuery = "DELETE FROM students WHERE id = ?";
PreparedStatement psDelete = con.prepareStatement(deleteQuery);
psDelete.setInt(1, 1);
psDelete.executeUpdate();
System.out.println("Record Deleted!");
// 8. Close Connection
con.close();

```

```
    }  
    catch (Exception e) {  
        System.out.println("Error : " + e);  
    }  
}  
}
```

Output

Connected to Database!

Record Inserted!

Reading Records:

1 | Alice | 20

Record Updated!

Record Deleted!

Result

Thus the above program to **C**onnect to a relational database (MySQL/PostgreSQL) and perform Create, Read, Update, and Delete operations using JDBC was executed successfully.

Ex. No : 05

Date:

Implement a simple servlet for uploading files to the server and downloading them using Multipart Config and File Output Stream.

Aim

To write a program to Implement a simple servlet for uploading files to the server and downloading them using Multipart Config and File Output Stream.

Algorithm

File Upload (POST Request)

1. Start the servlet and annotate with `@MultipartConfig`.
2. Get the server upload directory path.
3. Create the directory if it doesn't exist.
4. Get uploaded file using `request.getPart("file")`.
5. Extract filename using `getSubmittedFileName()`.
6. Open `InputStream` from the uploaded file.
7. Create a `FileOutputStream` to save the file on the server.
8. Read bytes from input stream and write to file.
9. Close streams and send success message.

File Download (GET Request)

1. Get filename from request (`?filename=name`).
2. Check if file exists in upload directory.
3. Set response headers (`Content-Disposition`) for download.
4. Open `FileInputStream` for the file.
5. Write file bytes to `response.getOutputStream()`.
6. Close streams.

Program

```
package com.example;

import java.io.*;

import javax.servlet.ServletException;

import javax.servlet.annotation.MultipartConfig;

import javax.servlet.annotation.WebServlet;
```

```

import javax.servlet.http.*;
@WebServlet("/fileServlet")
@MultipartConfig
public class FileServlet extends HttpServlet {
    private static final String UPLOAD_DIR = "uploads";
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Path to store uploaded files
        String uploadPath = getServletContext().getRealPath("") + File.separator + UPLOAD_DIR;
        // Create folder if it doesn't exist
        File uploadDir = new File(uploadPath);
        if (!uploadDir.exists()) {
            uploadDir.mkdir();
        }
        // Get uploaded file
        Part filePart = request.getPart("file");
        String fileName = filePart.getSubmittedFileName();
        // File path to save on server
        String filePath = uploadPath + File.separator + fileName;
        // Write file using FileOutputStream
        FileOutputStream fos = new FileOutputStream(filePath);
        InputStream is = filePart.getInputStream();
        byte[] buffer = new byte[1024];
        int bytesRead;
        while ((bytesRead = is.read(buffer)) != -1) {
            fos.write(buffer, 0, bytesRead);
        }
        fos.close();
        is.close();
        response.getWriter().println("File uploaded successfully: " + fileName);
    }
    @Override

```

```

protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String fileName = request.getParameter("filename");
    String filePath = getServletContext().getRealPath("") +
        File.separator + UPLOAD_DIR + File.separator + fileName;
    File file = new File(filePath);
    if (!file.exists()) {
        response.getWriter().println("File not found!");
        return;
    }
    // Set browser download properties
    response.setContentType("application/octet-stream");
    response.setHeader("Content-Disposition", "attachment;filename=" + fileName);
    // Read file using FileInputStream
    FileInputStream fis = new FileInputStream(file);
    OutputStream os = response.getOutputStream();
    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = fis.read(buffer)) != -1) {
        os.write(buffer, 0, bytesRead);
    }
    fis.close();
    os.close();
}
}

```

Upload Form (HTML / JSP)

```

<!DOCTYPE html>
<html>
<body>
<h2>Upload File</h2>

```

```
<form action="fileServlet" method="post" enctype="multipart/form-data">
  <input type="file" name="file"><br><br>
  <input type="submit" value="Upload">
</form>
<h2>Download File</h2>
<form action="fileServlet" method="get">
  Enter File Name: <input type="text" name="filename">
  <input type="submit" value="Download">
</form>
</body>
</html>
```

Output

File uploaded successfully: demo.txt

download → demo.txt

Result

Thus the above program to Implement a simple servlet for uploading files to the server and downloading them using Multipart Config and File Output Stream.

Ex. No : 06

Date:

Develop a servlet to handle HTTP GET and POST methods. Accept form data (e.g., user registration) and display it

Aim

To write a program to Develop a servlet to handle HTTP GET and POST methods. Accept form data (e.g., user registration) and display it

Algorithm

GET Request

1. User opens the registration form (HTML / JSP).
2. Form data is sent to servlet using GET (optional).
3. Servlet retrieves values using `request.getParameter()`.
4. Servlet displays the received values in HTML format.

POST Request

1. User fills the registration form and clicks **Submit**.
2. The form sends data using POST method.
3. In `doPost()` method, the servlet retrieves values using `request.getParameter()`.
4. Create an HTML response using `PrintWriter`.
5. Display user details (name, email, password, etc.).
6. End response.

Program

HTML Form (registration.html)

```
<!DOCTYPE html>
<html>
<body>
<h2>User Registration Form</h2>
<form action="RegisterServlet" method="post">
    Name: <input type="text" name="name"><br><br>
```

```
Email: <input type="email" name="email"><br><br>
Password: <input type="password" name="password"><br><br>
<input type="submit" value="Register">
</form>
</body>
</html>
```

Servlet Program (RegisterServlet.java)

```
package com.example;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/RegisterServlet")
public class RegisterServlet extends HttpServlet {

    // Handle GET request
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        out.println("<h2>Received GET Request</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
    }

    // Handle POST request
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
        throws ServletException, IOException {  
    response.setContentType("text/html");  
    PrintWriter out = response.getWriter();  
    String name = request.getParameter("name");  
    String email = request.getParameter("email");  
    String password = request.getParameter("password");  
    out.println("<h2>User Registration Successful</h2>");  
    out.println("Name: " + name + "<br>");  
    out.println("Email: " + email + "<br>");  
    out.println("Password: " + password + "<br>");  
    }  
}
```

Output

POST – Registration Submitted

```
User Registration Successful  
Name: Ram Kumar  
Email: ram@gmail.com  
Password: 12345
```

GET – Direct URL Example

URL:

```
http://localhost:8080/MyApp/RegisterServlet?name=John&email=john@gmail.com
```

Output:

```
Received GET Request  
Name: John  
Email: john@gmail.com
```

Result

Thus the above program to develop a servlet to handle HTTP GET and POST methods. Accept form data (e.g., user registration) and display it was executed successfully

Ex. No : 07

Date:

Design a responsive UI form using Bootstrap and React to capture user input and display the result

Aim

To write a program to Design a responsive UI form using Bootstrap and React to capture user input and display the result

Algorithm

1. Start a new React component.
2. Import Bootstrap CSS.
3. Create a form with Bootstrap classes (`container`, `form-group`, `btn`, etc.).
4. Create state variables using `useState()` to store user input.
5. Handle form submission using a function (`handleSubmit`).
6. Prevent page reload using `event.preventDefault()`.
7. Store submitted data in a separate state variable.
8. Display the submitted result inside a Bootstrap card.

Program

Simple React Program (Responsive UI Form using Bootstrap)

Save as: UserForm.js

```
import React, { useState } from "react";
import 'bootstrap/dist/css/bootstrap.min.css';
export default function UserForm() {
  const [name, setName] = useState("");
  const [result, setResult] = useState("");
  const handleSubmit = (event) => {
    event.preventDefault();
    setResult(name);
  };
  return (
    <div className="container mt-4">
      <h2 className="text-center mb-4">Responsive User Input Form</h2>
```

```

<form onSubmit={handleSubmit} className="card p-4 shadow">
  <div className="form-group mb-3">
    <label>Enter Your Name:</label>
    <input
      type="text"
      className="form-control"
      placeholder="Type here..."
      value={name}

      onChange={(e) => setName(e.target.value)}
    />
  </div>
  <button type="submit" className="btn btn-primary w-100">
    Submit
  </button>
</form>

{/* Display result */}
{result && (
  <div className="alert alert-success mt-4">
    <h4>You entered: {result}</h4>
  </div>
)}

</div>
);
}

```

Output

Before Submit

A clean Bootstrap form appears:

```
-----  
| Enter Your Name: [_____] |  
| [Submit]                |  
-----
```

After User Enters “Priya” and Clicks Submit

You entered: Priya

Displayed in a green **Bootstrap success alert**.

Result

Thus the above program to Design a responsive UI form using Bootstrap and React to capture user input and display the result was executed successfully.

Ex. No : 08

Date:

Create a JSP application using scripting elements, EL for data display, and JSTL for conditional and loop-based rendering.

Aim

To write a program to Create a JSP application using scripting elements, EL for data display, and JSTL for conditional and loop-based rendering.

Algorithm

1. Start Application

- User opens `index.jsp` containing a student registration form.
- JSP scripting elements show dynamic data (e.g., current date).

2. User Submits Form

- The form sends data (name, age, grade) to **StudentServlet** using POST method.

3. Processing in Servlet (Controller)

- Read form values using `request.getParameter()`.
- Create a **Student object**.
- Add it to a shared array list.
- Store list in `request` scope using `setAttribute()`.
- Forward to `display.jsp`.

4. Display Students

- `display.jsp` uses:
 - **EL** to access student object properties (`${stu.name}`)
 - **JSTL** `<c:forEach>` to loop through student list
 - **Scripting element** to generate row numbers

5. Show Output

- The page displays all students in a table.

Program

index.jsp (Uses Scripting Elements)

```
<%@ page language="java" import="java.util.Date" %>
<!DOCTYPE html>
<html>
<body>
<h2>Student Registration Form</h2>
<!-- JSP Scripting Element -->
<p>Current Date & Time: <%= new Date() %></p>
<form action="StudentServlet" method="post">
    Name : <input type="text" name="name"><br><br>
    Age  : <input type="number" name="age"><br><br>
    Grade: <input type="text" name="grade"><br><br>
    <input type="submit" value="Add Student">
</form>
</body>
</html>
```

Student.java (Model Class)

```
package com.example;

public class Student {
    private String name;
    private int age;
    private String grade;
    public Student(String name, int age, String grade) {
        this.name = name;
        this.age = age;
        this.grade = grade;
    }
}
```

```
public String getName() { return name; }  
public int getAge() { return age; }  
public String getGrade() { return grade; }  
}
```

StudentServlet.java (Controller)

```
package com.example;  
import java.io.IOException;  
import java.util.ArrayList;  
import javax.servlet.annotation.WebServlet;  
import javax.servlet.http.*;  
import javax.servlet.*;  
@WebServlet("/StudentServlet")  
public class StudentServlet extends HttpServlet {  
    ArrayList<Student> studentList = new ArrayList<>();  
    protected void doPost(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        String name = request.getParameter("name");  
        int age = Integer.parseInt(request.getParameter("age"));  
        String grade = request.getParameter("grade");  
        // Create Student object  
        Student student = new Student(name, age, grade);  
        studentList.add(student);  
        // Pass to JSP using request attribute  
        request.setAttribute("students", studentList);  
        // Forward to display.jsp  
        RequestDispatcher rd = request.getRequestDispatcher("display.jsp");  
        rd.forward(request, response);  
    }  
}
```

display.jsp (Uses *EL* + *JSTL* + *Scripting*)

```
<%@ page language="java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<body>
<h2>Student List</h2>
<!-- JSP scripting for row count -->
<% int count = 1; %>
<table border="1" cellpadding="10">
  <tr>
    <th>#</th>
    <th>Name</th>
    <th>Age</th>
    <th>Grade</th>
  </tr>
  <!-- JSTL Loop + Expression Language -->
  <c:forEach var="stu" items="{students}">
    <tr>
      <td><%= count++ %></td>
      <td>${stu.name}</td>
      <td>${stu.age}</td>
      <td>${stu.grade}</td>
    </tr>
  </c:forEach>
</table>
</body>
</html>
```

Output

Input Screen (index.jsp)

Student Registration Form

Current Date & Time: Tue Nov 14 11:20:23 IST 2025

Name: John

Age: 20

Grade: A

[Add Student]

Output Screen (display.jsp)

Student List

```
-----  
#  Name      Age      Grade  
1  John       20       A  
2  Priya      21       B  
3  Kavın      19       A+
```

Result

Thus the above program to create a JSP application using scripting elements, EL for data display, and JSTL for conditional and loop-based rendering was executed successfully.

Ex. No : 09

Date:

Build a Java application using Hibernate and JPA annotations to perform CRUD operations on a Student entity.

Aim

To write a program to simple Build a Java application using Hibernate and JPA annotations to perform CRUD operations on a Student entity

Algorithm

- Step 1: Start the program
- Step 2: Create Hibernate SessionFactory
- Step 3: Create a Student object
- Step 4: Save the Student into the database (CREATE)
- Step 5: Read the Student (READ)
- Step 6: Update the Student details (UPDATE)
- Step 7: Delete the Student (DELETE)
- Step 8: Close the session

Program

Student.java (Entity Class)

```
import jakarta.persistence.*;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private int marks;
    public Student() {}
    public Student(String name, int marks) {
        this.name = name;
```

```

        this.marks = marks;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getMarks() { return marks; }
    public void setMarks(int marks) { this.marks = marks; }
    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", marks=" + marks + "]";
    }
}

```

HibernateUtil.java

```

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
    private static SessionFactory sessionFactory;
    static {
        sessionFactory = new Configuration()
            .configure("hibernate.cfg.xml")
            .addAnnotatedClass(Student.class)
            .buildSessionFactory();
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}

```

MainApp.java (CRUD Operations)

```
import org.hibernate.Session;
import org.hibernate.Transaction;
public class MainApp {
    public static void main(String[] args) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();
        // CREATE
        Student s1 = new Student("Riya", 90);
        session.persist(s1);
        System.out.println("Student Saved: " + s1);
        // READ
        Student s2 = session.get(Student.class, s1.getId());
        System.out.println("Student Read: " + s2);

        // UPDATE
        s2.setMarks(95);
        session.merge(s2);
        System.out.println("Student Updated: " + s2);
        // DELETE
        session.remove(s2);
        System.out.println("Student Deleted with ID: " + s2.getId());
        tx.commit();
        session.close();
    }
}
```

hibernate.cfg.xml

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/testdb</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">root</property>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
    </session-factory>
</hibernate-configuration>
```

Output

Hibernate: insert into student (marks, name) values (?, ?)

Student Saved: Student [id=1, name=Riya, marks=90]

Hibernate: select student0_.id, student0_.marks, student0_.name from student student0_ where id=?

Student Read: Student [id=1, name=Riya, marks=90]

Hibernate: update student set marks=?, name=? where id=?

Student Updated: Student [id=1, name=Riya, marks=95]

Hibernate: delete from student where id=?

Student Deleted with ID: 1

Result

Thus the above program to simple Build a Java application using Hibernate and JPA annotations to perform CRUD operations on a Student entity was executed successfully.

Ex. No : 10

Date:

Mini-project integrating Spring and Hibernate framework.

Aim

To write a program to develop Mini-project integrating Spring and Hibernate framework.

Algorithm

Step 1: Start the application
Step 2: Spring loads its configuration file
Step 3: Spring creates Hibernate SessionFactory
Step 4: The user (main class) calls StudentService
Step 5: StudentService calls StudentDAO
Step 6: DAO performs CRUD using Hibernate Session
Step 7: Results return back to main class
Step 8: Display output

Program

Student Entity (JPA + Hibernate)

```
package com.project.entity;
import jakarta.persistence.*;
@Entity
@Table(name = "student")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private int marks;
    public Student() {}
    public Student(String name, int marks) {
        this.name = name;
```

```

        this.marks = marks;
    }
    // Getters / Setters
    public int getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getMarks() { return marks; }
    public void setMarks(int marks) { this.marks = marks; }

    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", marks=" + marks + "]";
    }
}

```

StudentDAO (Hibernate + Spring)

```

package com.project.dao;

import com.project.entity.Student;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

@Repository
public class StudentDAO {

    @Autowired
    private SessionFactory sessionFactory;

    public void save(Student s) {
        Session session = sessionFactory.getCurrentSession();
        session.persist(s);
    }
}

```

```

public Student get(int id) {
    return sessionFactory.getCurrentSession().get(Student.class, id);
}

public void update(Student s) {
    sessionFactory.getCurrentSession().merge(s);
}

public void delete(int id) {
    Student s = get(id);
    if (s != null) {
        sessionFactory.getCurrentSession().remove(s);
    }
}
}

```

StudentService (Spring Service Layer)

```

package com.project.service;

import com.project.dao.StudentDAO;
import com.project.entity.Student;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class StudentService {

    @Autowired
    private StudentDAO dao;

    @Transactional
    public void addStudent(Student s) { dao.save(s); }

    @Transactional
    public Student getStudent(int id) { return dao.get(id); }

    @Transactional

```



```

    public void updateStudent(Student s) { dao.update(s); }

    @Transactional

    public void deleteStudent(int id) { dao.delete(id); }

}

```

Spring Configuration (spring-config.xml)

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:tx="http://www.springframework.org/schema/tx"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd
           http://www.springframework.org/schema/tx
           http://www.springframework.org/schema/tx/spring-tx.xsd">
    <!-- Component Scan -->
    <context:component-scan base-package="com.project" />
    <!-- Load Hibernate config -->
    <bean id="sessionFactory"
          class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
        <property name="configLocation" value="classpath:hibernate.cfg.xml"/>
        <property name="packagesToScan" value="com.project.entity"/>
    </bean>
    <!-- Transaction Manager -->
    <bean id="txManager"
          class="org.springframework.orm.hibernate5.HibernateTransactionManager">
        <property name="sessionFactory" ref="sessionFactory"/>
    </bean>
    <tx:annotation-driven/>

```

</beans>

Hibernate Configuration (hibernate.cfg.xml)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<!DOCTYPE hibernate-configuration PUBLIC
```

```
"-//Hibernate/Hibernate Configuration DTD//EN"
```

```
"http://hibernate.org/dtd/hibernate-configuration-3.0.dtd">
```

```
<hibernate-configuration>
```

```
  <session-factory>
```

```
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
```

```
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/testdb</property>
```

```
    <property name="hibernate.connection.username">root</property>
```

```
    <property name="hibernate.connection.password">root</property>
```

```
    <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
```

```
    <property name="hibernate.show_sql">true</property>
```

```
    <property name="hibernate.hbm2ddl.auto">update</property>
```

```
  </session-factory>
```

```
</hibernate-configuration>
```

Output

Application Start

Spring Application Started...
Loading Spring Configuration...
SessionFactory created successfully.

When addStudent(new Student("Arun", 85)) is executed

Hibernate: insert into student (marks, name) values (?, ?)
Student Added Successfully!

When getStudent(1) is executed

Hibernate: select student0_.id as id1_0_0_,
student0_.marks as marks2_0_0_,
student0_.name as name3_0_0_
from student student0_
where student0_.id=?

Student Fetched: Student [id=1, name=Arun, marks=85]

When updateStudent() is executed

Hibernate: select student0_.id as id1_0_0_,
student0_.marks as marks2_0_0_,
student0_.name as name3_0_0_
from student student0_
where student0_.id=?

Hibernate: update student
set marks=?, name=?
where id=?

Student Updated Successfully!
Updated Data: Student [id=1, name=Arun, marks=95]

When deleteStudent(1) is executed

Hibernate: select student0_.id as id1_0_0_,
student0_.marks as marks2_0_0_,
student0_.name as name3_0_0_
from student student0_
where student0_.id=?

Hibernate: delete from student where id=?

Student Deleted Successfully!

Spring Application Started...

Adding Student...

Hibernate: insert into student (marks, name) values (?, ?)

Student Added Successfully!

Fetching Student...

Hibernate: select ... from student where id=?

Student Fetched: Student [id=1, name=Arun, marks=85]

Updating Student...

Hibernate: update student set marks=?, name=? where id=?

Student Updated Successfully!

Deleting Student...

Hibernate: delete from student where id=?

Student Deleted Successfully!

Result

Thus the above program to develop Mini-project integrating Spring and Hibernate framework was executed successfully.